

FMLY PRODUCTION

System Architecture & How It Works

1. Overview

FMLY Production is a private photo & video gallery platform built for managing casting shoots. Clients log in, browse their assigned galleries, like photos, add notes, and download shoot docs. Admins manage all content, users, and access from a single admin panel — no external CMS needed.

Live URLs

Website	<code>www.fmlyprod.com</code>
Media CDN	<code>media.fmlyprod.com</code>
Upload Worker	<code>fmly-upload.fmlyprod.workers.dev</code>
Source Code	<code>github.com/fmlyprod/fmlyprod</code>

2. Architecture — Four Moving Parts

GitHub Pages	<code>www.fmlyprod.com</code>	Hosts the single <code>index.html</code> file that is the entire app — admin panel + client gallery. Free, auto-deploys on every git push to main branch. No server required.
GitHub Repo (fmlyprod/fmlyprod)	Public repo, private data	Stores <code>index.html</code> , <code>clients.json</code> (users + tokens), manifest files (photo lists), <code>likes-log.json</code> (likes + notes). All writes go through the GitHub Contents API.
Cloudflare R2	<code>media.fmlyprod.com</code>	Object storage (like S3) for all photos and videos. Free tier: 10 GB storage, 10M reads/month. Serves files via CDN at <code>media.fmlyprod.com</code> . New uploads go here — no GitHub file size limits.
Cloudflare Worker	<code>fmly-upload.fmlyprod.workers.dev</code>	Serverless function that authenticates upload requests and issues presigned R2 URLs. Clients upload directly to R2 from the browser — no file passes through a server.

3. Key Data Files (all in GitHub repo)

File	Location / Format	Purpose
------	-------------------	---------

<code>clients.json</code>	Root of repo	Master config. Stores: all client usernames + passwords, which project each user belongs to, folder-level access permissions, shoot docs list per user, encoded GitHub write token (gt), encoded upload password (up).
<code>images/{project}/manifest.json</code>	e.g. images/bakery/manifest.json	Photo index for each project. Structure: { CATEGORY: { FOLDER: [url, url, ...] } }. Deleting a photo removes it from this list. Adding a folder creates a new key.
<code>likes-log.json</code>	Root of repo	One entry per user session. Stores: username, project, timestamp, array of liked photo URLs, and a notes map (url -> note text). Written by the client gallery on every like/note change.

4. Admin Panel (login: admin password)

Four tabs accessible after admin login at www.fmlyprod.com:

Tab	What It Does
GALLERY	Create and manage projects (Bakery, Dunkin, etc). Upload photos/videos to a project. Remove a project (photos stay on storage, project removed from clients.json).
PHOTOS	Browse any project's folder structure. View, delete individual photos, or delete entire folders. Deletes update the manifest — files remain on GitHub/R2 (hidden, not truly deleted).
USERS	Create client accounts. Set username, password, and project assignment. Control folder-level access (which specific folders each user can see). Upload and manage Shoot Docs (PDFs) per user.
LIKES LOG	View all liked photos per session, organized by user. See notes left on photos. Coincidences section shows photos liked by multiple users. Download a ZIP of all liked photos per session. Export an Excel file (columns: Picture, File Name, User, Note).

5. Client Gallery (what clients see)

Step	Detail
Login	Client enters username + password. System matches against clients.json. If allowed[] is set, only those folders are visible.
Browse	Folders shown by category. Client taps a folder to see all photos/videos inside.
Like ♥	Tap the heart on any photo. Liked photos appear in the LIKED tab. Likes sync to likes-log.json on GitHub every 5 seconds after last change.
Note ✎	Tap the pencil icon on any photo to type a note. Notes are saved locally and synced to GitHub alongside likes. Visible to admin in Likes Log.
Download	Each photo has a download button. Whole folder can be downloaded as a ZIP.
Shoot Docs	If the admin uploaded PDFs for this user, a SHOOT DOCS button appears at the top. Client can preview or download any PDF.

6. How Photo Upload Works

When an admin drops photos/videos into the uploader:

- Browser authenticates with the Cloudflare Worker using the admin password (stored encoded in clients.json).
- Worker returns a short-lived JWT token.
- For each file, browser requests a presigned R2 URL from the Worker.
- Browser uploads the file directly to R2 using the presigned URL — no middleman.
- On success, the CDN URL ([https://media.fmlyprod.com/...](https://media.fmlyprod.com/)) is added to the project manifest.
- Manifest is saved back to GitHub via the Contents API using the write token.

7. Tokens & Security

Item	Detail
GitHub Write Token	Stored XOR-encoded in clients.json as 'gt' field. Loaded on page init. Used for all writes (manifests, likes-log, clients.json). Never visible in JS source.
Upload Password	Stored XOR-encoded in clients.json as 'up' field. Used to authenticate with the Cloudflare Worker. Worker never exposes R2 credentials to browser.
Client Passwords	Stored in plain text in clients.json (low-sensitivity — these are gallery access codes, not personal accounts). Matched client-side on login.
upload-worker.js	The Cloudflare Worker source file containing all credentials is in .gitignore and NEVER committed to GitHub. Must be deployed manually via Cloudflare dashboard.

8. Running Costs

Service	Plan	Cost
GitHub Pages	Free public repo	\$0
Cloudflare R2	Free tier (10 GB, 10M reads/mo)	\$0
Cloudflare Workers	Free tier (100K req/day)	\$0
Domain (fmlyprod.com)	Wherever registered	~\$12/yr